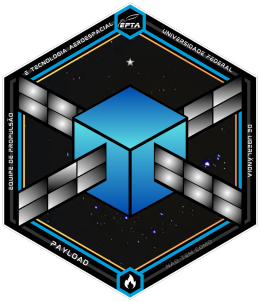
PAYLOAD

Desafio de Processo Seletivo
Payload

Universidade Federal
de Uberlândia



Gerentes

JOÃO VITOR FARIA SILVA
GABRIEL HENRIQUE SILVA CARDOSO

Equipe de Propulsão e Tecnologia Aeroespacial

11 de outubro de 2026

Introdução

A seguir é apresentado o desafio proposto para o processo seletivo da Payload. Ele consiste no desenvolvimento de um projeto completo de hardware e software embarcado: a escolha de componentes, o esquemático, o design da PCB e o firmware para um microcontrolador STM32. Ao final, deve ser enviada uma apresentação do trabalho desenvolvido juntamente dos itens requisitados. Pedimos que tudo o que for desenvolvido seja entregue, independente do candidato terminar ou não o desafio por completo.

Desafio: Datalogger Ambiental e Inercial com STM32.

A equipe está desenvolvendo um payload para voar em um foguete e precisa registrar, durante todo o voo, as condições do ambiente (temperatura, umidade e pressão) e o movimento do veículo (aceleração e velocidade angular). Os dados devem ficar gravados em um cartão microSD para análise posterior. O foco do trabalho é projetar a eletrônica dessa placa e o firmware que a controla, usando um microcontrolador da família STM32.

O valor real do projeto está menos na perfeição da placa e mais na capacidade de pesquisar, escolher componentes com critério e justificar cada decisão de projeto. O objetivo é demonstrar domínio das ferramentas de design eletrônico, organização no desenvolvimento e senso crítico de engenharia ao equilibrar custo, tamanho, consumo e facilidade de montagem.

Não esperamos que você entregue uma placa pronta para ir ao espaço. Esperamos ver como você aprende, busca informações em datasheets e documentações e estrutura seu raciocínio.

Recursos e Instruções:

O candidato tem livre escolha do software de design (KiCad, EasyEDA, Altium, etc.) e do ambiente de desenvolvimento do firmware (STM32CubeIDE, PlatformIO, etc.). Abaixo estão os requisitos fundamentais para nortear seu trabalho:

1. **Microcontrolador**: um STM32 para comandar o sistema (sugestão: STM32F411CEU6, que possui SDIO, USB e bastante memória flash).
2. **Sensor de temperatura, umidade e pressão**: comunicação via **I^2C** (sugestão: BME280).
3. **IMU**: acelerômetro e giroscópio com comunicação via **SPI** (sugestão: ICM-42688-P ou LSM6DSO).
4. **Armazenamento**: slot de cartão microSD, preferencialmente em SDIO de 4 bits, com gravação de arquivos CSV.
5. **Alimentação e programação**: entrada USB-C com regulador de 3,3 V e conector SWD para gravação e depuração.
6. **Documentação**: todo o projeto deve estar em um repositório público no GitHub.

Bloco	Componente sugerido	Interface
MCU	STM32F411CEU6	—
T/U/P	BME280	I^2C
IMU	ICM-42688-P / LSM6DSO	SPI + pino INT
Armazenamento	Slot microSD	SDIO 4 bits
Energia	USB-C + LDO 3,3 V	—
Depuração	Conector SWD	SWDIO / SWCLK

Nota: o foco é a viabilidade do projeto, levando em consideração a lógica das conexões e a facilidade de montagem. Os componentes sugeridos podem ser trocados por outros, desde que mantidos os protocolos I^2C e SPI. Dê preferência para componentes acessíveis e baratos.

Desafio em si:

Desenvolva um projeto que cubra os seguintes pontos:

1. **Seleção de Componentes**: escolha os componentes e justifique cada um (custo, disponibilidade, consumo, encapsulamento).
2. **Esquemático**: conecte os componentes, incluindo:
   - USB-C com proteção ESD e LDO de 3,3 V;
   - MCU com 100 nF por pino VDD, filtro no VDDA, cristal, BOOT0 e reset;
   - BME280 no I^2C com pull-ups, e IMU no SPI com os pinos SCK, MISO, MOSI, CS e INT;
   - slot microSD no SDIO com pull-ups e detecção de cartão;
   - SWD, LED de status, botão e pontos de teste.
3. **Design da PCB**: posicione os componentes e trace as trilhas, atentando para:
   - plano de GND contínuo e capacitores de desacoplamento junto aos pinos do MCU;
   - BME280 afastado do regulador e do MCU, porque o calor distorce a temperatura;
   - trilhas SDIO curtas e de comprimento parecido, e I^2C e SPI curtos;
   - DRC sem erros e arquivos de fabricação gerados (Gerbers, BOM e posicionamento).
4. **Firmware**: implemente a aquisição e o registro de dados:
   - configuração de I2C, SPI, SDIO com FatFS e timers;
   - driver do BME280 (leitura dos registradores e compensação) e do IMU (taxa, faixa e leitura por SPI);
   - gravação de um CSV com timestamp, sincronizando o arquivo periodicamente;
   - tratamento de erros: cartão ausente, falha de sensor e cartão cheio.
5. **Montagem e Validação**: se possível, monte a placa e teste em ordem (medição de 3,3 V, *blink*, scan I^2C, WHO_AM_I do IMU e criação de arquivo no SD). Compare as leituras com um instrumento de referência e verifique o IMU em repouso (aceleração próxima de 1 g). Caso não monte a placa, valide o firmware em uma Nucleo ou Black Pill com módulos em protoboard.
6. **Documentação no GitHub**: organize o repositório e escreva um README.

Estrutura sugerida do repositório:

`/hardware`	projeto de PCB, Gerbers, BOM e PDF do esquemático
`/firmware`	projeto do STM32CubeIDE
`/docs`	diagrama de blocos, fotos, testes e datasheets
`/scripts`	análise dos CSVs em Python
`README.md`	objetivo, pinagem, build e resultados

Entregáveis:

Uma apresentação rápida ou documento PDF contendo:

- A Lista de Materiais (BOM) com os componentes escolhidos e uma breve justificativa do porquê escolheu cada um.
- Imagem, arquivo ou link para o Esquemático.
- Imagem, arquivo ou link para o Design de PCB (pode ser a visualização 2D ou 3D do software).
- Código do firmware e instruções de build e gravação.
- Link para o repositório público no GitHub.
- Em caso de uso de datasheets dos componentes, cite-os no documento.
- Caso a placa seja montada: fotos, log de testes e um CSV real gravado no SD.

Extras:

Estes itens não são obrigatórios, mas contam pontos positivos na avaliação:

- Elabore o fluxograma/algoritmo de funcionamento do sistema.
- Apresente as principais bibliotecas utilizadas para o funcionamento do sistema.
- Saída de dados em tempo real por USB CDC e LED de status indicando gravação e erros.
- RTC com cristal de 32,768 kHz para timestamps absolutos.
- DMA no SPI do IMU e interrupção de *data-ready*, evitando polling.
- Filtro complementar ou Madgwick para estimar a inclinação.
- Modo de baixo consumo (Stop mode) com medição de corrente.
- Script em Python que lê o CSV e plota os dados.
- Release no GitHub com Gerbers, BOM e binário do firmware, e tags por marco.

Critérios de Avaliação:

Critério	O que é avaliado	Pontos
Esquemático	Correção, desacoplamento, organização e ERC limpo	15
Layout da PCB	Plano de GND, roteamento, posicionamento térmico e DRC limpo	20
Firmware	Drivers, estrutura do código e tratamento de erros	25
Funcionamento	Dados dos dois sensores gravados corretamente no SD	20
Documentação	README, histórico de commits, fotos e relatório de testes	15
Extras	Funcionalidades opcionais e análise dos dados	5
Total		**100**

Considerações Finais

Caso o candidato não consiga terminar o desafio, ele será avaliado com base no que foi desenvolvido. Por isso, reforçamos: não deixe de enviar seu trabalho. Em contrapartida, trabalho extra será valorizado.

Todo e qualquer material de referência utilizado no projeto deve ser citado no documento, seja essa referência bibliográfica, datasheet, vídeo, site, blog ou fórum.

Caso apareça alguma dúvida, não hesite em contatar algum dos gerentes responsáveis pela área:

- Gabriel Henrique Silva Cardoso – +55 64 98114-6682 (WhatsApp)
- João Vitor Faria Silva – +55 34 99192-2241 (WhatsApp)